

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
    roc_curve
)

from sklearn.model_selection import cross_val_score
```

```
df = pd.read_csv("adult_cleaned.csv")
print("Dataset Shape:", df.shape)
print("\nFirst 5 Rows:")
display(df.head())
```

Dataset Shape: (48813, 15)

First 5 Rows:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	
3	53	Private	234721	11th	7	Married-civ-	Handlers-cleaners	Husband	Black	Male	0	0	

```
print(df.info())
print(df.describe())
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
```

```
0 age 48813 non-null int64
1 workclass 48813 non-null object
2 fnlwgt 48813 non-null int64
3 education 48813 non-null object
4 education-num 48813 non-null int64
5 marital-status 48813 non-null object
6 occupation 48813 non-null object
7 relationship 48813 non-null object
8 race 48813 non-null object
9 sex 48813 non-null object
10 capital-gain 48813 non-null int64
11 capital-loss 48813 non-null int64
12 hours-per-week 48813 non-null int64
13 native-country 48813 non-null object
14 income 48813 non-null object
dtypes: int64(6), object(9)
memory usage: 5.6+ MB
None
age fnlwgt education-num capital-gain capital-loss \
count 48813.000000 4.881300e+04 48813.000000 48813.000000 48813.000000
```

```

75%      48.000000  2.576200e+05  12.000000  0.000000  0.000000
max      90.000000  1.490400e+06  16.000000  99999.000000  4356.000000

      hours-per-week
count    48813.000000
mean      40.425051
std       12.390954
min        1.000000
25%       40.000000
50%       40.000000
75%       45.000000
max       99.000000

```

Missing Values:

```

age      0
workclass  0
fnlwgt   0
education  0
education-num  0
marital-status  0
occupation  0
relationship  0
race      0
sex       0
capital-gain  0
capital-loss  0
hours-per-week  0
native-country  0
income     0
dtype: int64

```

Duplicate Rows:
0

```

df = df.apply(
    lambda col: col.str.strip() if col.dtype == "object" else col
)
print(df.isnull().sum())
df = df.replace("?", np.nan)
print(df.isnull().sum())
df = df.dropna().copy()
print("Shape after cleaning:", df.shape)

```

```

age      0
workclass  0
fnlwgt   0
education  0
education-num  0
marital-status  0
occupation  0
relationship  0
race      0
sex       0
capital-gain  0
capital-loss  0
hours-per-week  0
native-country  0
income     0
dtype: int64
age      0
workclass  0
fnlwgt   0
education  0
education-num  0
marital-status  0
occupation  0
relationship  0
race      0
sex       0
capital-gain  0
capital-loss  0
hours-per-week  0
native-country  0
income     0
dtype: int64
Shape after cleaning: (48813, 15)

```

```

duplicates = df.duplicated().sum()
print("Duplicate rows:", duplicates)
df = df.drop_duplicates().copy()
print("Shape after removing duplicates:", df.shape)

```

Duplicate rows: 0
Shape after removing duplicates: (48813, 15)

```

print(df["income"].value_counts())
df["income"] = (

```

```
df["income"]
    .str.replace(".", "", regex=False)
    .str.strip()
)
print(df["income"].value_counts())
```

```
income
<=50K      24698
<=50K.     12430
>50K        7839
>50K.       3846
Name: count, dtype: int64
income
<=50K      37128
>50K       11685
Name: count, dtype: int64
```

```
df["income"] = df["income"].map({
    "<=50K": 0,
    ">50K": 1
})
print(df["income"].value_counts())
print(df["income"].value_counts(normalize=True) * 100)
```

```
income
0      37128
1      11685
Name: count, dtype: int64
income
0      76.061705
1      23.938295
Name: proportion, dtype: float64
```

```
X = df.drop("income", axis=1)
y = df["income"]
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (48813, 14)
y shape: (48813,)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (39050, 14)
Testing data: (9763, 14)
```

```
numerical_features = X.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

categorical_features = X.select_dtypes(
    include=["object"]
).columns.tolist()

print("Numerical Features:")
print(numerical_features)

print("\nCategorical Features:")
print(categorical_features)
```

```
Numerical Features:
['age', 'fnlwtg', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']

Categorical Features:
['workclass', 'education', 'marital-status', 'occupation', 'relationship', 'race', 'sex', 'native-country']
```

```
preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            StandardScaler(),
            numerical_features
        ),
        (
```

```

        "cat",
        OneHotEncoder(
            handle_unknown="ignore"
        ),
        categorical_features
    )
]
)

```

```

logistic_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", LogisticRegression(
            max_iter=1000
        ))
    ]
)
logistic_model.fit(X_train, y_train)
y_pred_lr = logistic_model.predict(X_test)
y_prob_lr = logistic_model.predict_proba(X_test)[:, 1]

```

```

accuracy_lr = accuracy_score(y_test, y_pred_lr)

precision_lr = precision_score(y_test, y_pred_lr)

recall_lr = recall_score(y_test, y_pred_lr)

f1_lr = f1_score(y_test, y_pred_lr)

roc_auc_lr = roc_auc_score(y_test, y_prob_lr)
print("Logistic Regression Results")
print("-----")
print("Accuracy :", accuracy_lr)
print("Precision:", precision_lr)
print("Recall   :", recall_lr)
print("F1 Score  :", f1_lr)
print("ROC-AUC   :", roc_auc_lr)
print(
    classification_report(
        y_test,
        y_pred_lr
    )
)

```

```

Logistic Regression Results
-----
Accuracy : 0.8532213458977773
Precision: 0.7399150743099787
Recall   : 0.5964912280701754
F1 Score : 0.6605069888651979
ROC-AUC  : 0.9056754356577826

```

	precision	recall	f1-score	support
0	0.88	0.93	0.91	7426
1	0.74	0.60	0.66	2337
accuracy			0.85	9763
macro avg	0.81	0.77	0.78	9763
weighted avg	0.85	0.85	0.85	9763

```

rf_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "classifier",
            RandomForestClassifier(
                n_estimators=200,
                random_state=42,
                n_jobs=-1,
                class_weight="balanced"
            )
        )
    ]
)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
y_prob_rf = rf_model.predict_proba(X_test)[:, 1]

```

```

accuracy_rf = accuracy_score(y_test, y_pred_rf)

```

```

precision_rf = precision_score(y_test, y_pred_rf)

recall_rf = recall_score(y_test, y_pred_rf)

f1_rf = f1_score(y_test, y_pred_rf)

roc_auc_rf = roc_auc_score(y_test, y_prob_rf)

print("Random Forest Results")
print("-----")
print("Accuracy :", accuracy_rf)
print("Precision:", precision_rf)
print("Recall   :", recall_rf)
print("F1 Score :", f1_rf)
print("ROC-AUC  :", roc_auc_rf)
print(
    classification_report(
        y_test,
        y_pred_lr
    )
)

```

```

Random Forest Results
-----
Accuracy : 0.8546553313530677
Precision: 0.732994923857868
Recall   : 0.6178861788617886
F1 Score : 0.6705363361968888
ROC-AUC  : 0.9040160448877939

```

	precision	recall	f1-score	support
0	0.88	0.93	0.91	7426
1	0.74	0.60	0.66	2337
accuracy			0.85	9763
macro avg	0.81	0.77	0.78	9763
weighted avg	0.85	0.85	0.85	9763

```

cm_lr = confusion_matrix(y_test, y_pred_lr)

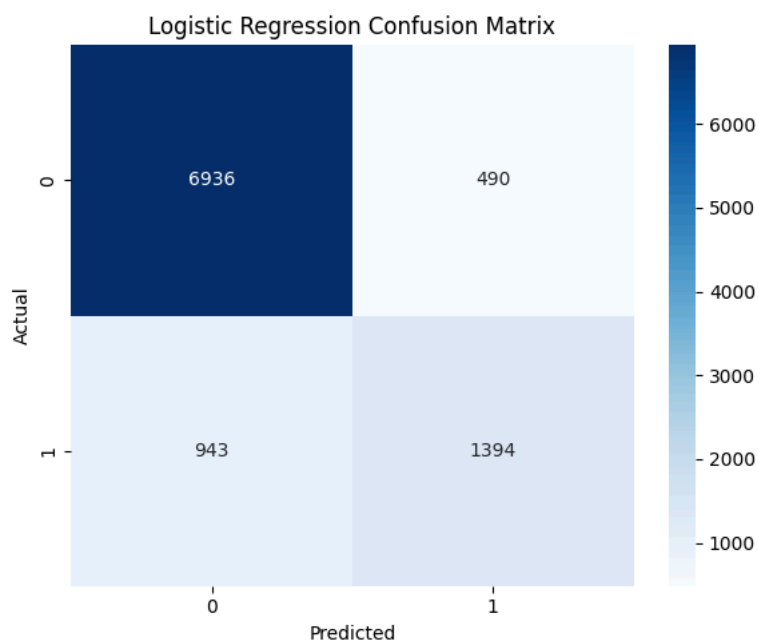
plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_lr,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Logistic Regression Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.tight_layout()
plt.show()

```



```

fpr_lr, tpr_lr, _ = roc_curve(
    y_test,
    y_prob_lr
)

fpr_rf, tpr_rf, _ = roc_curve(
    y_test,
    y_prob_rf
)

plt.figure(figsize=(8, 6))

plt.plot(
    fpr_lr,
    tpr_lr,
    label=f"Logistic Regression (AUC = {roc_auc_lr:.3f})"
)

plt.plot(
    fpr_rf,
    tpr_rf,
    label=f"Random Forest (AUC = {roc_auc_rf:.3f})"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

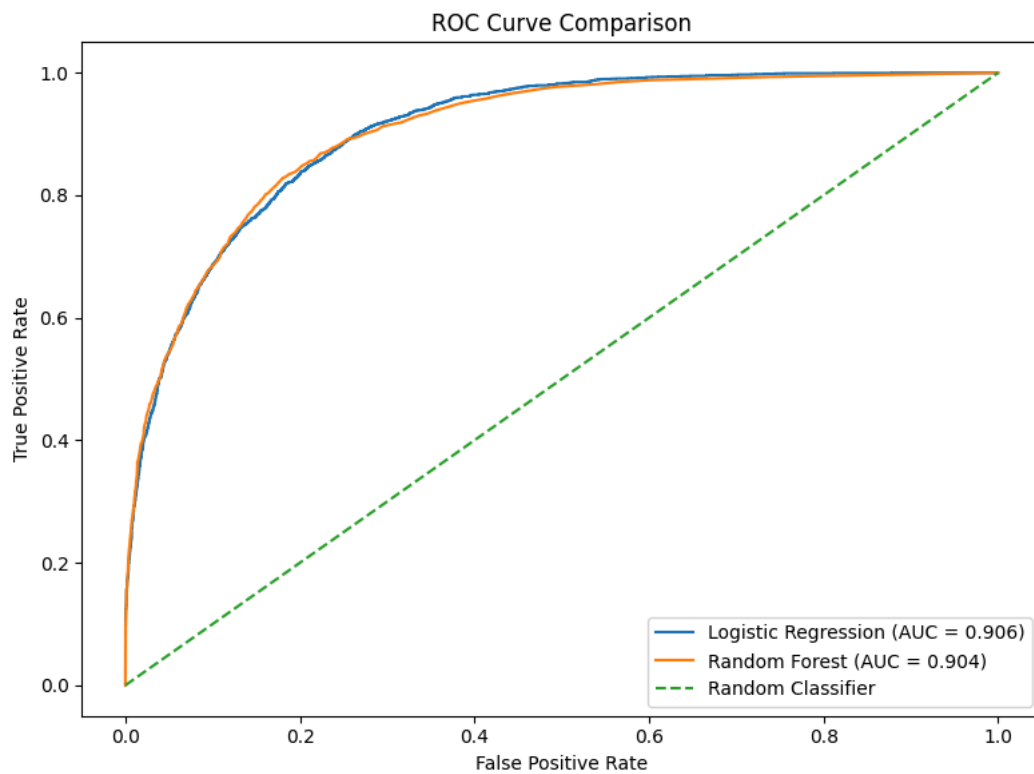
plt.title("ROC Curve Comparison")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

plt.legend()

plt.tight_layout()
plt.show()

```



```

results = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Random Forest"
    ],
    "Accuracy": [
        accuracy_lr,
        accuracy_rf
    ]
})

```

```

    ],
    "Precision": [
        precision_lr,
        precision_rf
    ],
    "Recall": [
        recall_lr,
        recall_rf
    ],
    "F1 Score": [
        f1_lr,
        f1_rf
    ],
    "ROC-AUC": [
        roc_auc_lr,
        roc_auc_rf
    ]
}
})
print(results)

```

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	Logistic Regression	0.853221	0.739915	0.596491	0.660507	0.905675
1	Random Forest	0.854655	0.732995	0.617886	0.670536	0.904016

```

cv_scores = cross_val_score(
    rf_model,
    X_train,
    y_train,
    cv=5,
    scoring="f1"
)

print("Cross-Validation F1 Scores:")
print(cv_scores)

print("\nMean CV F1 Score:")
print(cv_scores.mean())

print("\nStandard Deviation:")
print(cv_scores.std())

```

Cross-Validation F1 Scores:
[0.66140351 0.65886483 0.66570188 0.65663717 0.66914924]

Mean CV F1 Score:
0.6623513264771843

Standard Deviation:
0.0045418819317613

```

preprocessor_fitted = rf_model.named_steps["preprocessor"]

feature_names = (
    preprocessor_fitted
    .get_feature_names_out()
)

importance = (
    rf_model
    .named_steps["classifier"]
    .feature_importances_
)

feature_importance = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importance
})

feature_importance = (
    feature_importance
    .sort_values(
        "Importance",
        ascending=False
    )
    .head(15)
)

```

```

plt.figure(figsize=(10, 6))

sns.barplot(
    data=feature_importance,

```

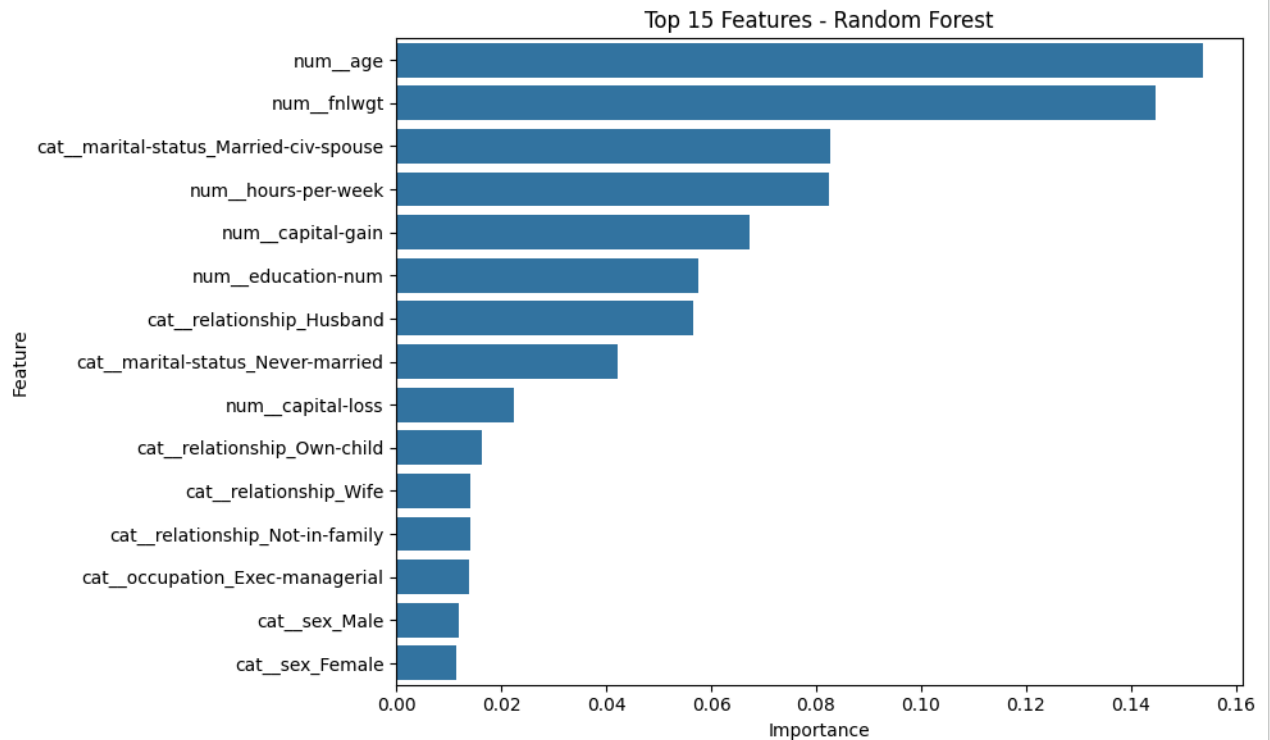
```

    x="Importance",
    y="Feature"
)

plt.title(
    "Top 15 Features - Random Forest"
)

plt.tight_layout()
plt.show()

```



```

errors = X_test.copy()

errors["Actual"] = y_test
errors["Predicted"] = y_pred_rf

misclassified = errors[
    errors["Actual"] != errors["Predicted"]
]

print(
    "Number of misclassified records:",
    len(misclassified)
)

print(
    misclassified[
        [
            "age",
            "education",
            "occupation",
            "hours-per-week",
            "Actual",
            "Predicted"
        ]
    ].head(20)
)

```

Number of misclassified records: 1419

	age	education	occupation	hours-per-week	Actual	Predicted
40831	48	Some-college	Adm-clerical	40	1	0
6679	34	HS-grad	Sales	40	0	1
34306	41	HS-grad	Exec-managerial	40	0	1
7731	42	Some-college	Tech-support	40	1	0
372	40	Assoc-voc	Machine-op-inspct	38	1	0
24552	46	Bachelors	Prof-specialty	45	1	0
22353	38	Bachelors	Exec-managerial	60	0	1
17449	33	HS-grad	Sales	45	0	1
33520	30	Bachelors	Adm-clerical	40	1	0
25151	27	Bachelors	Sales	45	0	1
32002	50	Some-college	Craft-repair	40	1	0
15756	36	HS-grad	Adm-clerical	40	1	0

32054	32	Assoc-voc	Other-service	60	1	0
6696	36	Bachelors	Prof-specialty	45	0	1
28582	44	HS-grad	Sales	35	1	0
12320	46	HS-grad	Sales	45	1	0
33430	36	HS-grad	Prof-specialty	60	1	0
11227	66	HS-grad	Sales	10	1	0
25256	36	Bachelors	Prof-specialty	45	1	0
32409	41	Bachelors	Prof-specialty	50	0	1